HR Attrition Prediction (Classification)

Problem Statement

Employee attrition is a critical issue for organizations, leading to increased hiring costs,productivity loss, and reduced employee morale.

The goal is to build a Machine Learning classification model that predicts whether an employee will leave the company(Attrition: Yes/No)

Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

Load Data

```
data=pd.read_csv('HR-Employee-Attrition.csv')
data
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Employ
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	
...	...	...	...	...	...	...	...	...	...	
1465	36	No	Travel_Frequently	884	Research & Development	23	2	Medical	1	
1466	39	No	Travel_Rarely	613	Research & Development	6	1	Medical	1	
1467	27	No	Travel_Rarely	155	Research & Development	4	3	Life Sciences	1	
1468	49	No	Travel_Frequently	1023	Sales	2	3	Medical	1	
1469	34	No	Travel_Rarely	628	Research & Development	8	3	Medical	1	

1470 rows × 35 columns

Understanding Variables that are needed for classification

Age: The age of the employee.

Attrition: The attrition status of the employee (binary: yes/no).

BusinessTravel: Frequency of business travel (e.g., non-travel, travel frequently, etc.).

DistanceFromHome: The distance from the employee's home to the workplace.

EnvironmentSatisfaction: The level of satisfaction with the work environment.

Gender: The gender of the employee.

JobRole: The role or position of the employee.

JobSatisfaction: The level of job satisfaction reported by the employee.

MaritalStatus: The marital status of the employee.

MonthlyIncome: The monthly income of the employee.

OverTime: Indicates if the employee works overtime (binary: yes/no).

PercentSalaryHike: The percentage increase in salary for the employee.

PerformanceRating: The performance rating of the employee.

RelationshipSatisfaction: The level of satisfaction with relationships at work.

StockOptionLevel: The level of stock options awarded to the employee.

TotalWorkingYears: The total number of years the employee has worked.

YearsAtCompany: The number of years the employee has worked at the current company.

YearsWithCurrManager: The number of years the employee has been with the current manager.

Basic Checks

```
data.shape
```

```
(1470, 35)
```

```
data.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	EmployeeNu
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	

5 rows × 35 columns

```
data.tail()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Employ
1465	36	No	Travel_Frequently	884	Research & Development	23	2	Medical	1	
1466	39	No	Travel_Rarely	613	Research & Development	6	1	Medical	1	
1467	27	No	Travel_Rarely	155	Research & Development	4	3	Life Sciences	1	
1468	49	No	Travel_Frequently	1023	Sales	2	3	Medical	1	
1469	34	No	Travel_Rarely	628	Research & Development	8	3	Medical	1	

5 rows × 35 columns

```
data.info()
data.columns
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
```

```
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',
       'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',
       'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
       'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
       'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
       'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',
       'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
       'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
       'YearsWithCurrManager'],
      dtype='object')
```

```
data.describe()
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRat
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0	1470.000000	1470.000000	1470.00000
mean	36.923810	802.485714	9.192517	2.912925	1.0	1024.865306	2.721769	65.89115
std	9.135373	403.509100	8.106864	1.024165	0.0	602.024335	1.093082	20.32942
min	18.000000	102.000000	1.000000	1.000000	1.0	1.000000	1.000000	30.00000
25%	30.000000	465.000000	2.000000	2.000000	1.0	491.250000	2.000000	48.00000
50%	36.000000	802.000000	7.000000	3.000000	1.0	1020.500000	3.000000	66.00000
75%	43.000000	1157.000000	14.000000	4.000000	1.0	1555.750000	4.000000	83.75000
max	60.000000	1499.000000	29.000000	5.000000	1.0	2068.000000	4.000000	100.00000

8 rows × 26 columns

```
print(data.dtypes)
```

```
Age                         int64
Attrition                  object
BusinessTravel             object
DailyRate                   int64
Department                 object
DistanceFromHome            int64
Education                   int64
EducationField             object
EmployeeCount               int64
EmployeeNumber              int64
EnvironmentSatisfaction     int64
Gender                     object
HourlyRate                  int64
JobInvolvement              int64
JobLevel                    int64
JobRole                    object
JobSatisfaction             int64
MaritalStatus              object
MonthlyIncome               int64
MonthlyRate                 int64
NumCompaniesWorked          int64
Over18                     object
OverTime                   object
PercentSalaryHike           int64
PerformanceRating           int64
RelationshipSatisfaction    int64
StandardHours               int64
StockOptionLevel            int64
TotalWorkingYears           int64
TrainingTimesLastYear       int64
WorkLifeBalance             int64
YearsAtCompany              int64
YearsInCurrentRole          int64
YearsSinceLastPromotion     int64
```

```
YearsWithCurrManager         int64
dtype: object
```

```
num_col= data.select_dtypes(include=['int64','float64'])
num_col
```

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount	EmployeeNumber	EnvironmentSatisfaction	HourlyRate	JobInvolve
0	41	1102	1	2	1	1	2	94	
1	49	279	8	1	1	2	3	61	
2	37	1373	2	2	1	4	4	92	
3	33	1392	3	4	1	5	4	56	
4	27	591	2	1	1	7	1	40	
...	...	...	...	...	...	...	...	...	
1465	36	884	23	2	1	2061	3	41	
1466	39	613	6	1	1	2062	4	42	
1467	27	155	4	3	1	2064	2	87	
1468	49	1023	2	3	1	2065	4	63	
1469	34	628	8	3	1	2068	2	82	

1470 rows × 26 columns

```
cat_cols = data.select_dtypes(include=['object'])
cat_cols
```

	Attrition	BusinessTravel	Department	EducationField	Gender	JobRole	MaritalStatus	Over18	OverTime
0	Yes	Travel_Rarely	Sales	Life Sciences	Female	Sales Executive	Single	Y	Yes
1	No	Travel_Frequently	Research & Development	Life Sciences	Male	Research Scientist	Married	Y	No
2	Yes	Travel_Rarely	Research & Development	Other	Male	Laboratory Technician	Single	Y	Yes
3	No	Travel_Frequently	Research & Development	Life Sciences	Female	Research Scientist	Married	Y	Yes
4	No	Travel_Rarely	Research & Development	Medical	Male	Laboratory Technician	Married	Y	No
...	...	...	...	...	...	...	...	...	...
1465	No	Travel_Frequently	Research & Development	Medical	Male	Laboratory Technician	Married	Y	No
1466	No	Travel_Rarely	Research & Development	Medical	Male	Healthcare Representative	Married	Y	No
1467	No	Travel_Rarely	Research &	Life Sciences	Male	Manufacturing	Married	Y	Yes

Next steps: Generate code with cat_cols New interactive sheet

```
for x in cat_cols:
  print(x,'--->',data[x].unique())
```

```
Attrition ---> ['Yes' 'No']
BusinessTravel ---> ['Travel_Rarely' 'Travel_Frequently' 'Non-Travel']
Department ---> ['Sales' 'Research & Development' 'Human Resources']
EducationField ---> ['Life Sciences' 'Other' 'Medical' 'Marketing' 'Technical Degree'
 'Human Resources']
Gender ---> ['Female' 'Male']
JobRole ---> ['Sales Executive' 'Research Scientist' 'Laboratory Technician'
 'Manufacturing Director' 'Healthcare Representative' 'Manager'
 'Sales Representative' 'Research Director' 'Human Resources']
MaritalStatus ---> ['Single' 'Married' 'Divorced']
Over18 ---> ['Y']
OverTime ---> ['Yes' 'No']
```

```
for x in cat_cols:
  print(data[x].value_counts())
```

```
Attrition
No     1233
Yes     237
Name: count, dtype: int64
BusinessTravel
Travel_Rarely        1043
Travel_Frequently     277
Non-Travel            150
```

```
Name: count, dtype: int64
Department
Research & Development    961
Sales                     446
Human Resources            63
Name: count, dtype: int64
EducationField
Life Sciences       606
Medical             464
Marketing           159
Technical Degree    132
Other                82
Human Resources      27
Name: count, dtype: int64
Gender
Male      882
Female    588
Name: count, dtype: int64
JobRole
Sales Executive              326
Research Scientist           292
Laboratory Technician        259
Manufacturing Director       145
Healthcare Representative    131
Manager                      102
Sales Representative          83
Research Director             80
Human Resources               52
Name: count, dtype: int64
MaritalStatus
Married     673
Single      470
Divorced    327
Name: count, dtype: int64
Over18
Y    1470
Name: count, dtype: int64
OverTime
No     1054
Yes     416
Name: count, dtype: int64
```

```
print(data.nunique())
```

```
Age                          43
Attrition                     2
BusinessTravel                3
DailyRate                   886
Department                    3
DistanceFromHome             29
Education                     5
EducationField                6
EmployeeCount                 1
EmployeeNumber             1470
EnvironmentSatisfaction       4
Gender                        2
HourlyRate                   71
JobInvolvement                4
JobLevel                      5
JobRole                       9
JobSatisfaction               4
MaritalStatus                 3
MonthlyIncome              1349
MonthlyRate                1427
NumCompaniesWorked           10
Over18                        1
OverTime                      2
PercentSalaryHike            15
PerformanceRating             2
RelationshipSatisfaction      4
StandardHours                 1
StockOptionLevel              4
TotalWorkingYears            40
TrainingTimesLastYear         7
WorkLifeBalance               4
YearsAtCompany               37
YearsInCurrentRole           19
YearsSinceLastPromotion      16
YearsWithCurrManager         18
dtype: int64
```

Data Cleaning

Check for Missng values

```
data.isnull().sum().sum()
```

```
np.int64(0)
```

Check for duplicates

```
data.duplicated().sum()
```

```
np.int64(0)
```

```
data.columns
```

```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'EducationField', 'EmployeeCount',
       'EmployeeNumber', 'EnvironmentSatisfaction', 'Gender', 'HourlyRate',
       'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
       'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
       'Over18', 'OverTime', 'PercentSalaryHike', 'PerformanceRating',
       'RelationshipSatisfaction', 'StandardHours', 'StockOptionLevel',
       'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
       'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
       'YearsWithCurrManager'],
      dtype='object')
```

```
cols_to_drop = ['EmployeeNumber','Over18','StandardHours', 'EmployeeCount']

data.drop(columns=cols_to_drop,inplace=True)
```

```
data.columns
```

```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'EducationField',
       'EnvironmentSatisfaction', 'Gender', 'HourlyRate', 'JobInvolvement',
       'JobLevel', 'JobRole', 'JobSatisfaction', 'MaritalStatus',
       'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked', 'OverTime',
       'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction',
       'StockOptionLevel', 'TotalWorkingYears', 'TrainingTimesLastYear',
       'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole',
       'YearsSinceLastPromotion', 'YearsWithCurrManager'],
      dtype='object')
```

```
num_cols= data.select_dtypes(include=['int64','float64'])
num_cols
```

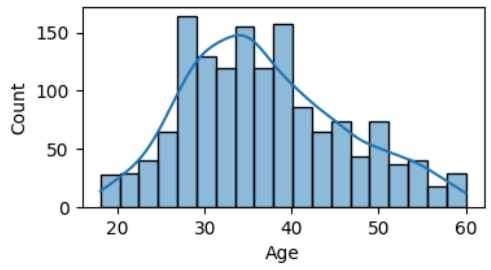
	Age	DailyRate	DistanceFromHome	Education	EnvironmentSatisfaction	HourlyRate	JobInvolvement	JobLevel	JobSatisfactior
0	41	1102	1	2	2	94	3	2	4
1	49	279	8	1	3	61	2	2	2
2	37	1373	2	2	4	92	2	1	3
3	33	1392	3	4	4	56	3	1	3
4	27	591	2	1	1	40	3	1	2
...	...	...	...	...	...	...	...	...	...
1465	36	884	23	2	3	41	4	2	4
1466	39	613	6	1	4	42	2	3	1
1467	27	155	4	3	2	87	4	2	2
1468	49	1023	2	3	4	63	2	2	2
1469	34	628	8	3	2	82	4	2	3

1470 rows × 23 columns

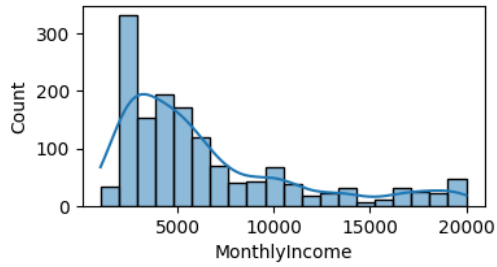
Exploratory data analysis

Univariate

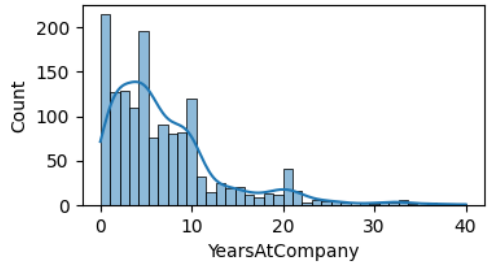
```
plt.figure(figsize=(4,2))
sns.histplot(data,x='Age',kde=True)
plt.show()
```



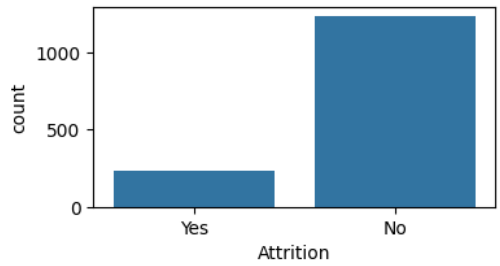

```
plt.figure(figsize=(4,2))
sns.histplot(data,x='MonthlyIncome',kde=True)
plt.show()
```



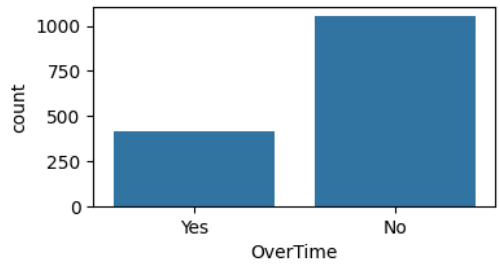

```
plt.figure(figsize=(4,2))
sns.histplot(data,x='YearsAtCompany',kde=True)
plt.show()
```



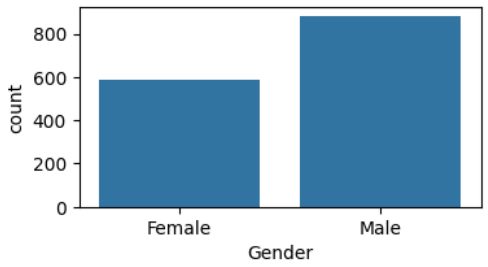

```
plt.figure(figsize=(4,2))
sns.countplot(data,x='Attrition')
plt.show()
```



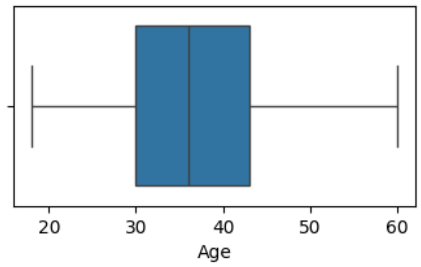

```
plt.figure(figsize=(4,2))
sns.countplot(data,x='OverTime')
plt.show()
```



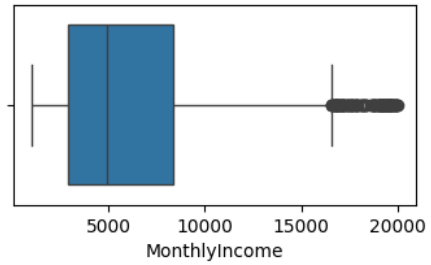

```
plt.figure(figsize=(4,2))
sns.countplot(data,x='Gender')
plt.show()
```



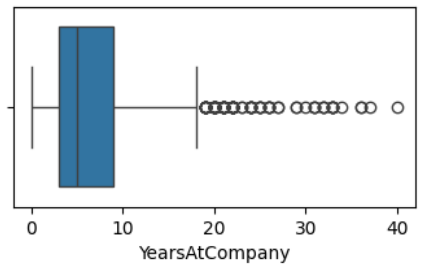

```
plt.figure(figsize=(4,2))
sns.boxplot(data,x='Age')
plt.show()
```



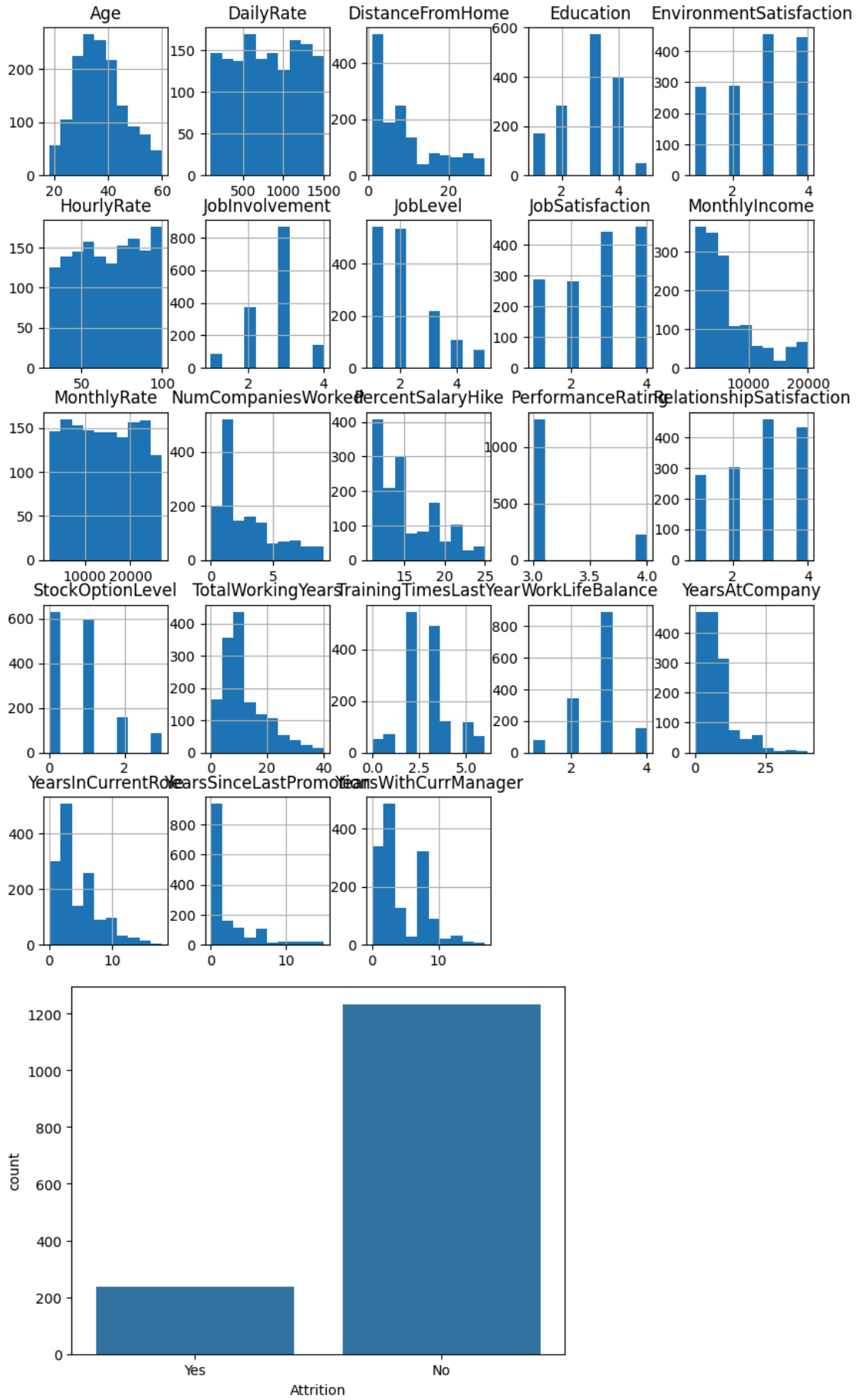

```
plt.figure(figsize=(4,2))
sns.boxplot(data,x='MonthlyIncome')
plt.show()
```




```
plt.figure(figsize=(4,2))
sns.boxplot(data,x='YearsAtCompany')
plt.show()
```

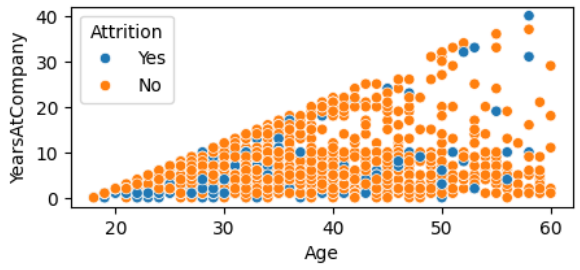



```
data.hist(figsize=(10,12))
plt.show()
sns.countplot(data,x='Attrition')
plt.show()
```

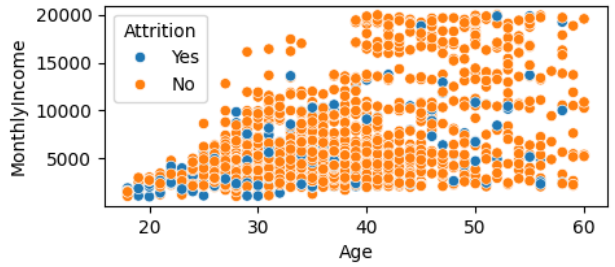


Bivariate Analysis

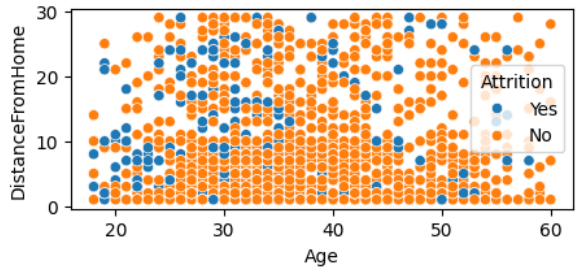
```
plt.figure(figsize=(5,2))
sns.scatterplot(data,x='Age',y='YearsAtCompany',hue='Attrition')
plt.show()
```



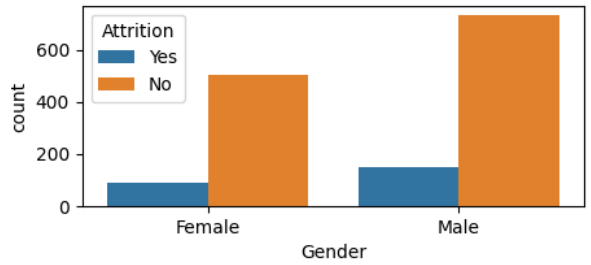

```
plt.figure(figsize=(5,2))
sns.scatterplot(data,x='Age',y='MonthlyIncome',hue='Attrition')
plt.show()
```



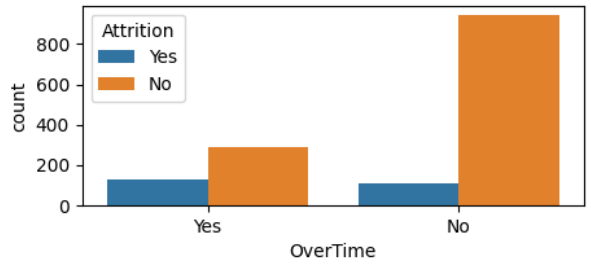

```
plt.figure(figsize=(5,2))
sns.scatterplot(data,x='Age',y='DistanceFromHome',hue='Attrition')
plt.show()
```




```
plt.figure(figsize=(5,2))
sns.countplot(data,x='Gender',hue='Attrition')
plt.show()
```

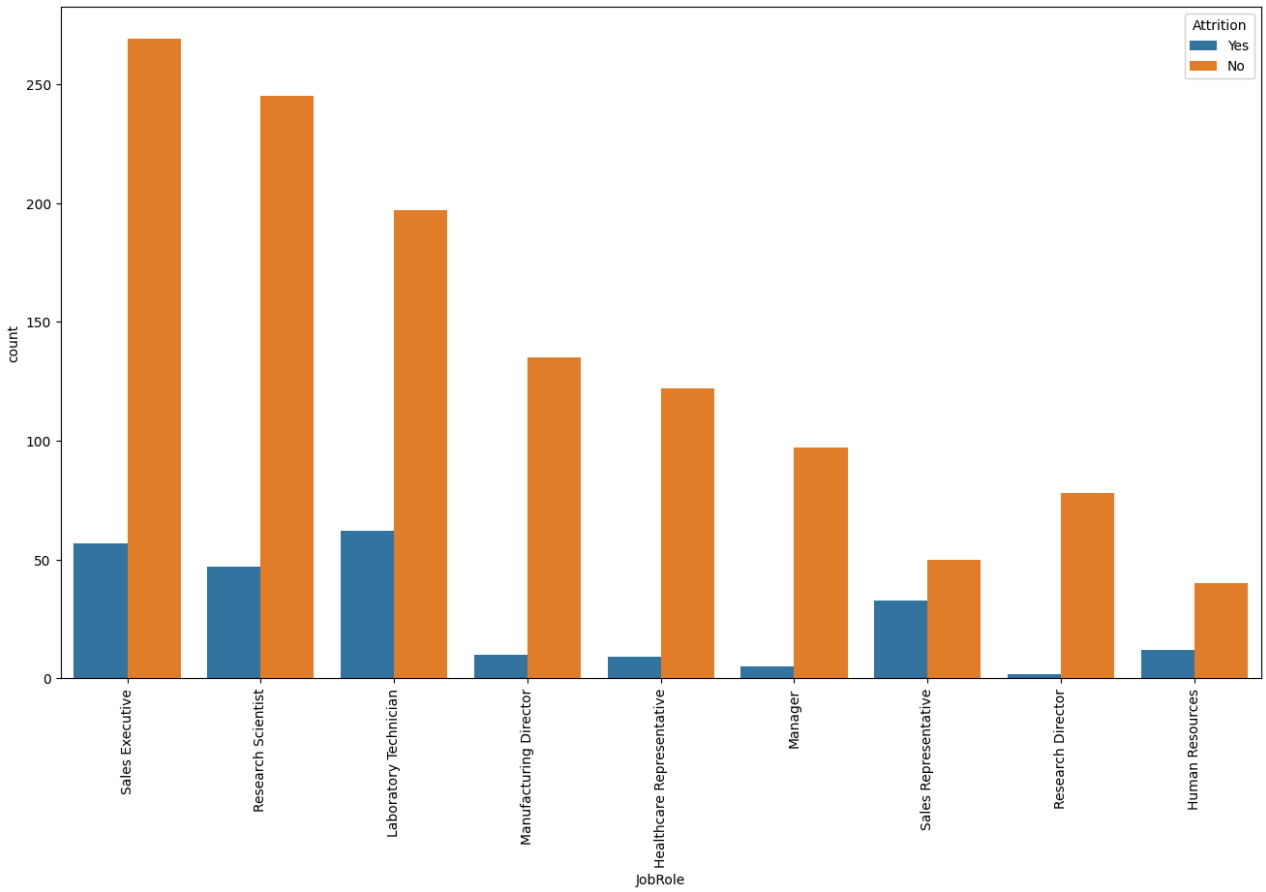



```
plt.figure(figsize=(5,2))
sns.countplot(data,x='OverTime',hue='Attrition')
plt.show()
```



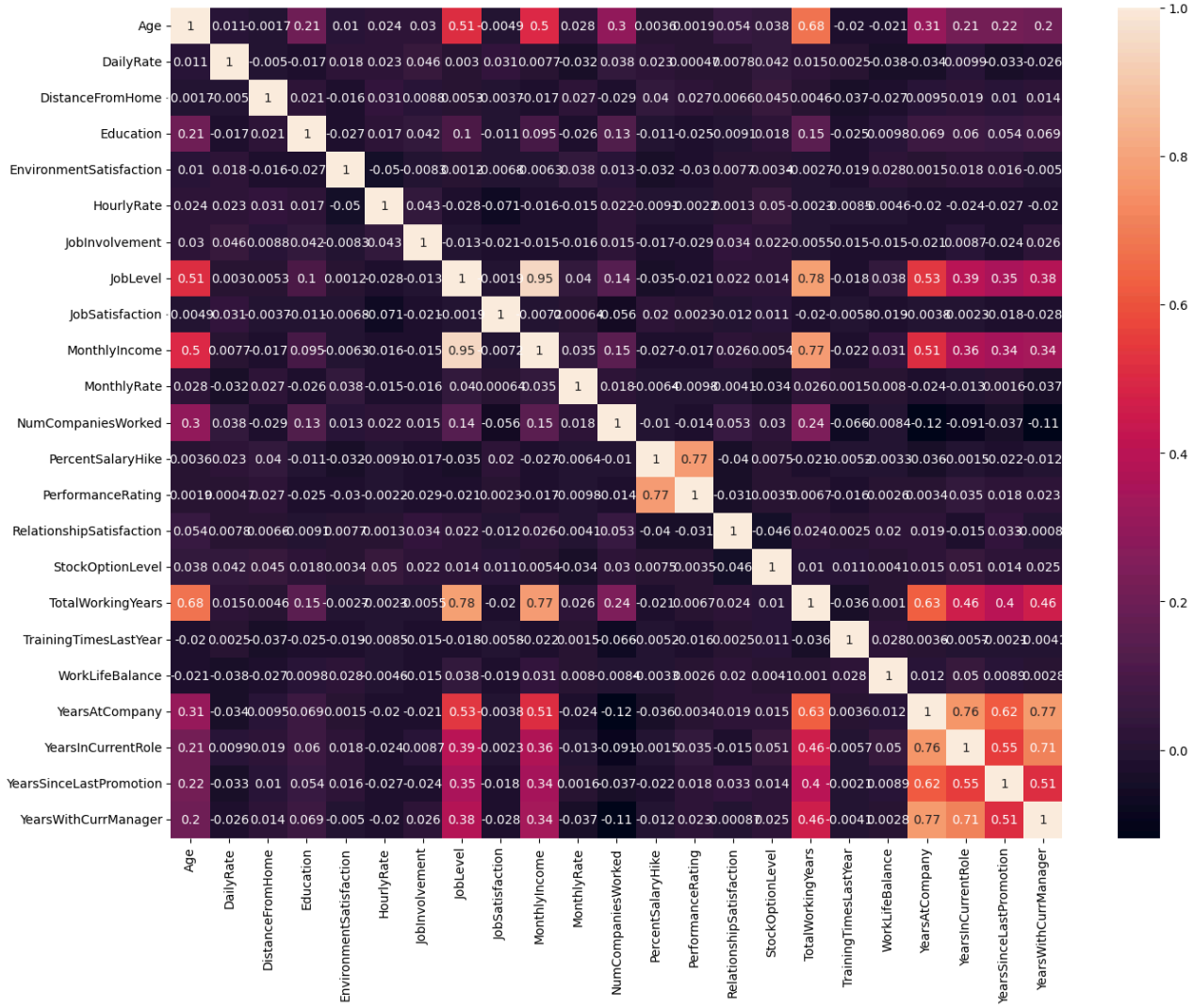

```
plt.figure(figsize=(16,9))
sns.countplot(data,x='JobRole',hue='Attrition')
```

```
plt.xticks(rotation=90)
plt.show()
```



Pairplot

```
plt.figure(figsize=(16,12))
sns.heatmap(num_cols.corr(),annot=True)
plt.show()
```

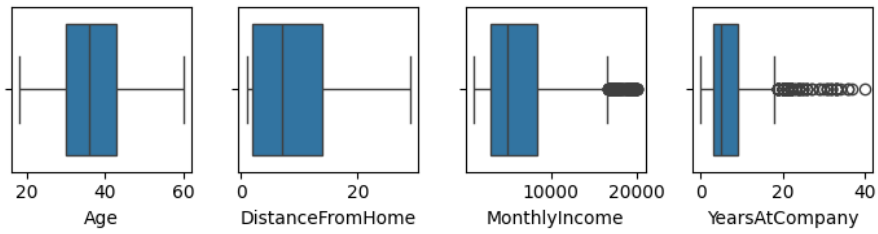



```
data.columns
```

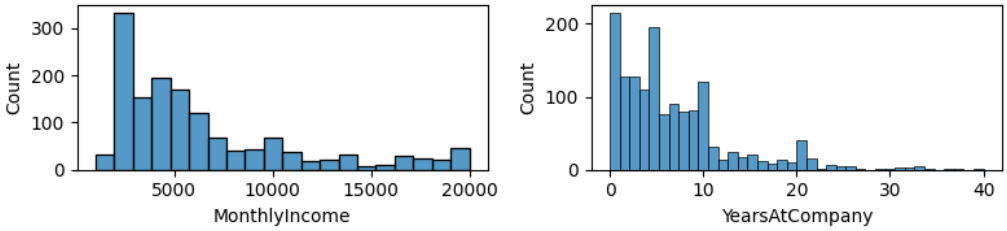
```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'EducationField',
       'EnvironmentSatisfaction', 'Gender', 'HourlyRate', 'JobInvolvement',
       'JobLevel', 'JobRole', 'JobSatisfaction', 'MaritalStatus',
       'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked', 'OverTime',
       'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction',
       'StockOptionLevel', 'TotalWorkingYears', 'TrainingTimesLastYear',
       'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole',
       'YearsSinceLastPromotion', 'YearsWithCurrManager'],
      dtype='object')
```

Handling Outliers

```
fig,ax=plt.subplots(1,4,figsize=(7,2))
sns.boxplot(data,x='Age',ax=ax[0])
sns.boxplot(data,x='DistanceFromHome',ax=ax[1])
sns.boxplot(data,x='MonthlyIncome',ax=ax[2])
sns.boxplot(data,x='YearsAtCompany',ax=ax[3])
plt.tight_layout()
```




```
fig,ax=plt.subplots(1,2,figsize=(8,2))
sns.histplot(data,x='MonthlyIncome',ax=ax[0])
sns.histplot(data,x='YearsAtCompany',ax=ax[1])
plt.tight_layout()
```




```
data.loc[:,'MonthlyIncome'].skew()
```

```
np.float64(1.3698166808390662)
```

```
data.loc[:,'YearsAtCompany'].skew()
```

```
np.float64(1.7645294543422085)
```

```
Q1 = data.loc[:,'MonthlyIncome'].quantile(0.25)
Q3 = data.loc[:,'MonthlyIncome'].quantile(0.75)
IQR = Q3-Q1
lower_limit= Q1 - 1.5 * IQR
upper_limit= Q3 + 1.5*IQR
```

```
# filter all the values greater than upper and less than lower
data[(data.loc[:,'MonthlyIncome']>upper_limit)|(data.loc[:,'MonthlyIncome']<lower_limit)]
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfacti
25	53	No	Travel_Rarely	1282	Research & Development	5	3	Other	
29	46	No	Travel_Rarely	705	Sales	2	4	Marketing	
45	41	Yes	Travel_Rarely	1360	Research & Development	12	3	Technical Degree	
62	50	No	Travel_Rarely	989	Research & Development	7	2	Medical	
105	59	No	Non-Travel	1420	Human Resources	2	4	Human Resources	
...	...	...	...	...	...	...	...	...	
1374	58	No	Travel_Rarely	605	Sales	21	3	Life Sciences	
1377	49	No	Travel_Frequently	1064	Research & Development	2	1	Life Sciences	
1401	55	No	Travel_Rarely	189	Human Resources	26	4	Human Resources	
1437	39	No	Non-Travel	105	Research & Development	9	3	Life Sciences	
1443	42	No	Travel_Rarely	300	Research & Development	2	3	Life Sciences	

114 rows × 31 columns

```
len(data[(data.loc[:,'MonthlyIncome']>upper_limit)|(data.loc[:,'MonthlyIncome']<lower_limit)])/len(data)*100
```

```
7.755102040816326
```

Since % of outliers is more than 5% will leave them as it is.

```
Q1 = data.loc[:,'YearsAtCompany'].quantile(0.25)
Q3 = data.loc[:,'YearsAtCompany'].quantile(0.75)
IQR = Q3-Q1
lower_limit= Q1 - 1.5 * IQR
upper_limit= Q3 + 1.5*IQR
```

```
# filter all the values greater than upper and less than lower
data[(data.loc[:,'YearsAtCompany']>upper_limit)|(data.loc[:,'YearsAtCompany']<lower_limit)]
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfactio
18	53	No	Travel_Rarely	1219	Sales	2	4	Life Sciences	
28	44	No	Travel_Rarely	477	Research & Development	7	4	Medical	
45	41	Yes	Travel_Rarely	1360	Research & Development	12	3	Technical Degree	
62	50	No	Travel_Rarely	989	Research & Development	7	2	Medical	
63	59	No	Travel_Rarely	1435	Sales	25	3	Life Sciences	
...	...	...	...	...	...	...	...	...	
1403	39	No	Travel_Rarely	119	Sales	15	4	Marketing	
1404	42	No	Non-Travel	335	Research & Development	23	2	Life Sciences	
1443	42	No	Travel_Rarely	300	Research & Development	2	3	Life Sciences	
1445	41	No	Travel_Rarely	582	Research & Development	28	4	Life Sciences	
1462	39	No	Travel_Rarely	722	Sales	24	1	Marketing	

104 rows × 31 columns

```
len(data[(data.loc[:,'YearsAtCompany']>upper_limit)|(data.loc[:,'YearsAtCompany']<lower_limit)])/len(data)*100
```

```
7.07482993197279
```

Since % of outliers is more than 5% will leave them as it is.

Encoding

```
from sklearn.preprocessing import LabelEncoder
scaler = LabelEncoder()
data.loc[:,'attrition']=scaler.fit_transform(data.loc[:,'Attrition'])
data.loc[:,'overtime']=scaler.fit_transform(data.loc[:,'OverTime'])
```

```
data = pd.get_dummies(data,columns=['Gender'],dtype='int')
```

```
data
```

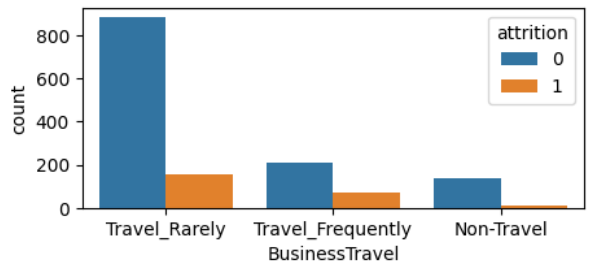
	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfacti
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	
...	...	...	...	...	...	...	...	...	
1465	36	No	Travel_Frequently	884	Research & Development	23	2	Medical	
1466	39	No	Travel_Rarely	613	Research & Development	6	1	Medical	
1467	27	No	Travel_Rarely	155	Research & Development	4	3	Life Sciences	
1468	49	No	Travel_Frequently	1023	Sales	2	3	Medical	
1469	34	No	Travel_Rarely	628	Research & Development	8	3	Medical	

1470 rows × 34 columns

```
data.columns
```

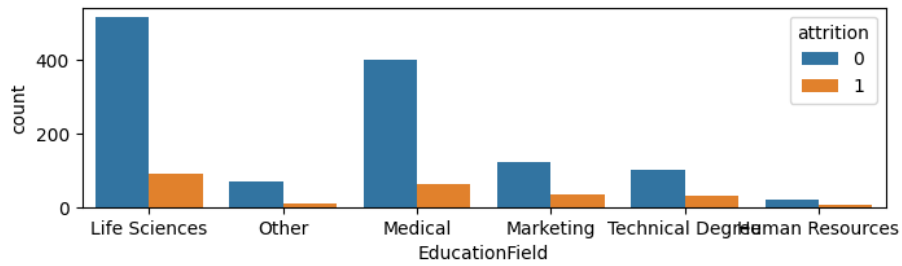
```
Index(['Age', 'Attrition', 'BusinessTravel', 'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'EducationField',
       'EnvironmentSatisfaction', 'HourlyRate', 'JobInvolvement', 'JobLevel',
       'JobRole', 'JobSatisfaction', 'MaritalStatus', 'MonthlyIncome',
       'MonthlyRate', 'NumCompaniesWorked', 'OverTime', 'PercentSalaryHike',
       'PerformanceRating', 'RelationshipSatisfaction', 'StockOptionLevel',
       'TotalWorkingYears', 'TrainingTimesLastYear', 'WorkLifeBalance',
       'YearsAtCompany', 'YearsInCurrentRole', 'YearsSinceLastPromotion',
       'YearsWithCurrManager', 'attrition', 'overtime', 'Gender_Female',
       'Gender_Male'],
      dtype='object')
```

```
plt.figure(figsize=(5,2))
sns.countplot(data,x='BusinessTravel',hue='attrition')
plt.show()
```




```
# use map
data.loc[:,'BusinessTravel']=data.loc[:,'BusinessTravel'].map({'Travel_Rarely':2,'Travel_Frequently':1,'Non-Travel':0})
```

```
plt.figure(figsize=(8,2))
sns.countplot(data,x='EducationField',hue='attrition')
plt.show()
```



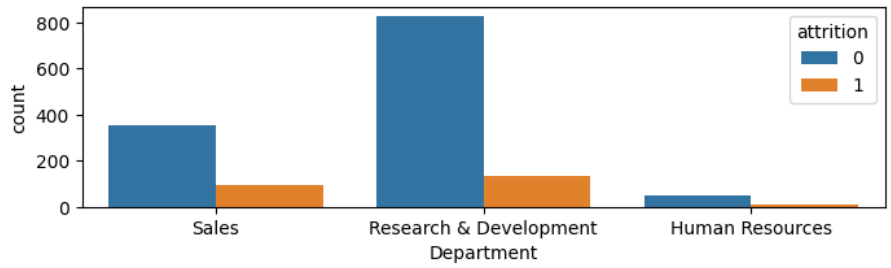
```
# use map
data.loc[:,'EducationField']=data.loc[:,'EducationField'].map({'Life Sciences':5,'Medical':4,'Marketing':3,'Technical Degree':2
```

```
data
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfactio
0	41	Yes	2	1102	Sales	1	2	5	
1	49	No	1	279	Research & Development	8	1	5	
2	37	Yes	2	1373	Research & Development	2	2	1	
3	33	No	1	1392	Research & Development	3	4	5	
4	27	No	2	591	Research & Development	2	1	4	
...	...	...	...	...	...	...	...	...	
1465	36	No	1	884	Research & Development	23	2	4	
1466	39	No	2	613	Research & Development	6	1	4	
1467	27	No	2	155	Research & Development	4	3	5	
1468	49	No	1	1023	Sales	2	3	4	
1469	34	No	2	628	Research & Development	8	3	4	

1470 rows × 34 columns

```
plt.figure(figsize=(8,2))
sns.countplot(data,x='Department',hue='attrition')
plt.show()
```



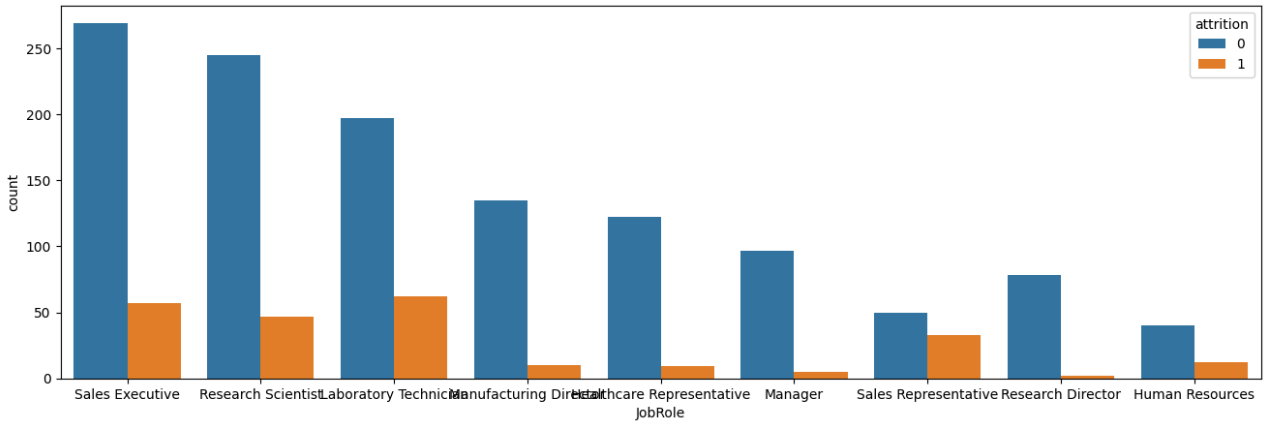

```
# use map
data.loc[:,'Department']=data.loc[:,'Department'].map({'Research & Development':2,'Sales':1,'Human Resources':0})
```

```
data
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfactior
0	41	Yes	2	1102	1	1	2	5	2
1	49	No	1	279	2	8	1	5	3
2	37	Yes	2	1373	2	2	2	1	4
3	33	No	1	1392	2	3	4	5	4
4	27	No	2	591	2	2	1	4	1
...	...	...	...	...	...	...	...	...	...
1465	36	No	1	884	2	23	2	4	3
1466	39	No	2	613	2	6	1	4	4
1467	27	No	2	155	2	4	3	5	2
1468	49	No	1	1023	1	2	3	4	4
1469	34	No	2	628	2	8	3	4	2

1470 rows × 34 columns

```
plt.figure(figsize=(16,5))
sns.countplot(data,x='JobRole',hue='attrition')
plt.show()
```




```
# use map
data.loc[:,'JobRole']=data.loc[:,'JobRole'].map({'Laboratory Technician':8,'Sales Executive':7,'Research Scientist':6,'Sales Re
```

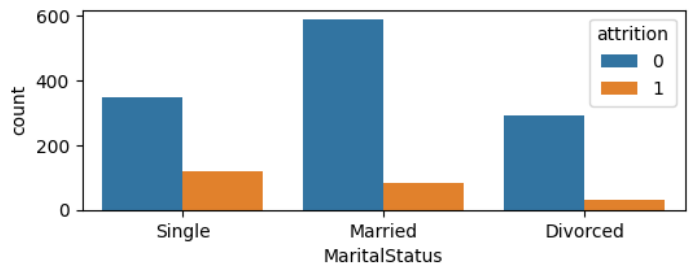
```
data.head()
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	H
0	41	Yes	2	1102	1	1	2	5	2	
1	49	No	1	279	2	8	1	5	3	
2	37	Yes	2	1373	2	2	2	1	4	
3	33	No	1	1392	2	3	4	5	4	
4	27	No	2	591	2	2	1	4	1	

5 rows × 34 columns

```
data.drop(columns=['Attrition'],inplace=True)
data.drop(columns=['OverTime'],inplace=True)
```

```
plt.figure(figsize=(6,2))
sns.countplot(data,x='MaritalStatus',hue='attrition')
plt.show()
```




```
# use map
data.loc[:,'MaritalStatus']=data.loc[:,'MaritalStatus'].map({'Single':2,'Married':1,'Divorced':0})
```

Scaling

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
data[['Age']]=scaler.fit_transform(data[['Age']])
data[['DistanceFromHome']]=scaler.fit_transform(data[['DistanceFromHome']])
data[['MonthlyIncome']]=scaler.fit_transform(data[['MonthlyIncome']])
data[['YearsAtCompany']]=scaler.fit_transform(data[['YearsAtCompany']])
```

```
data
```

	Age	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EnvironmentSatisfaction	Hou
0	0.446350	2	1102	1	-1.010909	2	5	2	
1	1.322365	1	279	2	-0.147150	1	5	3	
2	0.008343	2	1373	2	-0.887515	2	1	4	
3	-0.429664	1	1392	2	-0.764121	4	5	4	
4	-1.086676	2	591	2	-0.887515	1	4	1	
...	...	...	...	...	...	...	...	...	
1465	-0.101159	1	884	2	1.703764	2	4	3	
1466	0.227347	2	613	2	-0.393938	1	4	4	
1467	-1.086676	2	155	2	-0.640727	3	5	2	
1468	1.322365	1	1023	1	-0.887515	3	4	4	
1469	-0.320163	2	628	2	-0.147150	3	4	2	

1470 rows × 32 columns

```
data.columns
```

```
Index(['Age', 'BusinessTravel', 'DailyRate', 'Department', 'DistanceFromHome',
       'Education', 'EducationField', 'EnvironmentSatisfaction', 'HourlyRate',
       'JobInvolvement', 'JobLevel', 'JobRole', 'JobSatisfaction',
       'MaritalStatus', 'MonthlyIncome', 'MonthlyRate', 'NumCompaniesWorked',
       'PercentSalaryHike', 'PerformanceRating', 'RelationshipSatisfaction',
       'StockOptionLevel', 'TotalWorkingYears', 'TrainingTimesLastYear',
       'WorkLifeBalance', 'YearsAtCompany', 'YearsInCurrentRole',
       'YearsSinceLastPromotion', 'YearsWithCurrManager', 'attrition',
       'overtime', 'Gender_Female', 'Gender_Male'],
      dtype='object')
```

```
print(data.dtypes)
```

```
Age                        float64
BusinessTravel              object
DailyRate                    int64
Department                  object
DistanceFromHome           float64
Education                    int64
EducationField              object
EnvironmentSatisfaction      int64
HourlyRate                   int64
JobInvolvement               int64
JobLevel                     int64
JobRole                     object
JobSatisfaction              int64
MaritalStatus               object
MonthlyIncome              float64
MonthlyRate                  int64
NumCompaniesWorked           int64
PercentSalaryHike            int64
PerformanceRating            int64
RelationshipSatisfaction     int64
StockOptionLevel             int64
TotalWorkingYears            int64
TrainingTimesLastYear        int64
WorkLifeBalance              int64
YearsAtCompany             float64
YearsInCurrentRole           int64
YearsSinceLastPromotion      int64
YearsWithCurrManager         int64
attrition                    int64
overtime                     int64
Gender_Female                int64
Gender_Male                  int64
dtype: object
```

```
data['overtime'] = data['overtime'].astype(int)
data['attrition'] = data['attrition'].astype(int)
data['BusinessTravel'] = data['BusinessTravel'].astype(int)
data['MaritalStatus']=data['MaritalStatus'].astype(int)
```

```
data['MaritalStatus']=data['MaritalStatus'].astype(int)
data['JobRole']=data['JobRole'].astype(int)
data['EducationField']=data['EducationField'].astype(int)
data['Department']=data['Department'].astype(int)
```

Split data for x and y

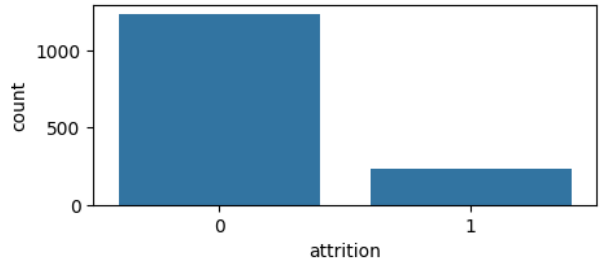
```
x = data.drop(columns='attrition')
y= data['attrition']
```

Split data for training and testing

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test = train_test_split(x,y,test_size=0.2,random_state=42)
```

Balancing data

```
plt.figure(figsize=(5,2))
sns.countplot(data,x='attrition')
plt.show()
```




```
from imblearn.over_sampling import SMOTE
from collections import Counter
smote = SMOTE(random_state=42)
x_sampled,y_sampled = smote.fit_resample(x_train,y_train)
# Balancing should be done only on training data
```

```
from sklearn.linear_model import LogisticRegression
model = LogisticRegression(max_iter=6000)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

```
model.intercept_
```

```
array([4.87308483])
```

```
model.predict_proba(x_test)
```

```
       [3.61630380e-01, 6.38369620e-01],
       [9.94784306e-01, 5.21569417e-03],
       [7.89422078e-01, 2.10577922e-01],
       [6.29726375e-01, 3.70273625e-01],
       [8.89516016e-01, 1.10483984e-01],
       [8.65665930e-01, 1.34334070e-01],
       [3.92419046e-01, 6.07580954e-01],
       [8.16573690e-01, 1.83426310e-01],
       [9.96132678e-01, 3.86732176e-03],
       [8.78850556e-01, 1.21149444e-01],
       [7.88608050e-01, 2.11391950e-01],
       [4.51128366e-01, 5.48871634e-01],
       [8.17040169e-01, 1.82959831e-01],
       [9.98580301e-01, 1.41969894e-03],
       [6.11302391e-01, 3.88697609e-01],
       [2.07413189e-01, 7.92586811e-01],
       [8.79222417e-01, 1.20777583e-01],
       [6.52876953e-01, 3.47123047e-01],
       [6.06645254e-01, 3.93354746e-01],
       [8.80374653e-01, 1.19625347e-01],
       [9.67928219e-01, 3.20717813e-02],
       [8.90076733e-01, 1.09923267e-01],
       [9.41212612e-01, 5.87873883e-02],
       [3.26835643e-01, 6.73164357e-01],
       [3.50458548e-01, 6.49541452e-01],
       [9.69371618e-01, 3.06283819e-02],
       [8.96020692e-01, 1.03979308e-01],
       [9.58936060e-01, 4.10639404e-02],
       [7.70363074e-01, 2.29636926e-01],
       [3.13162555e-01, 6.86837445e-01],
       [9.96163970e-01, 3.83602958e-03],
       [7.57976619e-01, 2.42023381e-01],
       [9.86353001e-01, 1.36469990e-02],
       [8.82432420e-01, 1.17567580e-01],
       [8.45821260e-01, 1.54178740e-01],
       [7.59641647e-01, 2.40358353e-01]])
```

Evaluate the model

```
from sklearn.metrics import confusion_matrix,accuracy_score,recall_score,precision_score,f1_score
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[223  32]
 [ 19  20]]
Accuracy:  0.826530612244898
Recall:  0.5128205128205128
Precision:  0.38461538461538464
f1_score:  0.43956043956043955
```

AUC ROC curve

```
from sklearn.metrics import roc_curve,roc_auc_score
```

```
# prob
y_prob=model.predict_proba(x_test)[:,1]
y_prob
```

```
       4.22217723e-01, 7.39781932e-01, 1.50720805e-02, 4.85252895e-01,
       8.44477628e-02, 7.21249786e-01, 4.44307296e-01, 5.70781634e-02,
       7.84018557e-01, 1.44980364e-01, 9.38498325e-02, 1.81912921e-03,
       1.36504349e-02, 2.28703265e-01, 1.62491580e-01, 2.11188057e-01,
       5.34669364e-05, 4.41087150e-01, 5.97783234e-02, 5.97490985e-01,
```

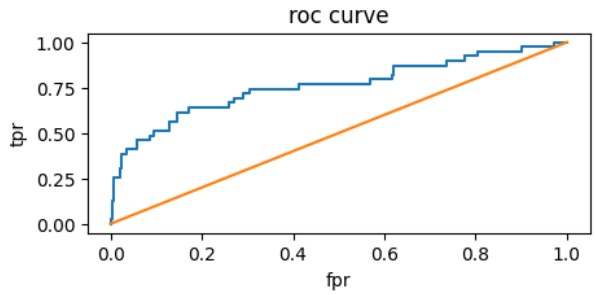
```
       3.14967264e-01, 8.89452239e-01, 1.55490745e-01, 1.70521804e-01,
       1.19502580e-01, 6.85371615e-01, 3.51043442e-01, 4.93799469e-01,
       2.31253198e-01, 2.76673412e-01, 1.11337497e-03, 5.75715251e-02,
       3.67229036e-02, 1.57578397e-01, 6.00324094e-02, 5.53064580e-01,
       1.09528324e-02, 5.08209769e-01, 6.03916608e-01, 2.26890913e-02,
       1.76968769e-01, 7.29738678e-03, 1.09412908e-01, 5.65975029e-02,
       7.38556379e-01, 8.87419784e-01, 2.12017896e-02, 2.67988875e-01,
       9.16217369e-02, 3.74497541e-02, 1.81476092e-01, 5.41532979e-01,
       9.29477174e-02, 1.65115241e-01, 4.64102690e-04, 8.92148293e-03,
       2.18787561e-02, 2.95674192e-02, 1.40201452e-01, 2.20279491e-02,
       2.32155721e-02, 9.39396078e-03, 9.97460021e-04, 1.77596190e-01,
       3.63956813e-01, 2.06905228e-01, 7.86815939e-02, 4.22507334e-01,
       4.87093542e-01, 2.38083705e-02, 2.23752622e-02, 9.56840008e-02,
       7.03573059e-01, 1.10094748e-01, 3.80233919e-01, 5.44472362e-02,
       1.26448813e-03, 2.98611279e-01, 1.71450535e-01, 2.72409394e-01,
       1.29495092e-01, 5.08077922e-02, 1.18673327e-01, 9.43220852e-02,
       9.31668207e-02, 3.06813124e-01, 6.48642568e-02, 7.58893814e-02,
       8.75414819e-02, 1.24178395e-02, 9.30275216e-01, 1.32444594e-03,
       3.10966231e-01, 2.40206599e-01, 8.09571312e-03, 8.90848974e-01,
       2.27170465e-01, 3.95776310e-01, 1.03485492e-01, 1.47276261e-01,
       5.20667141e-01, 8.97468201e-02, 5.12063139e-01, 4.69558652e-01,
       3.41251236e-02, 4.71676442e-01, 5.12818857e-01, 5.96925115e-03,
       9.48425175e-03, 8.16045044e-02, 3.89637487e-02, 6.25293332e-03,
       6.13308602e-01, 8.19372710e-02, 5.78331686e-01, 3.65750144e-03,
       5.83151023e-03, 1.32337020e-01, 1.03376263e-01, 4.57093828e-01,
       4.99683205e-02, 9.46185483e-02, 4.56585795e-01, 1.85291400e-01,
       6.03330890e-01, 2.40105954e-02, 8.70519093e-02, 3.22370334e-03,
       3.41267903e-01, 4.68067144e-01, 6.38369620e-01, 5.21569417e-03,
       2.10577922e-01, 3.70273625e-01, 1.10483984e-01, 1.34334070e-01,
       6.07580954e-01, 1.83426310e-01, 3.86732176e-03, 1.21149444e-01,
       2.11391950e-01, 5.48871634e-01, 1.82959831e-01, 1.41969894e-03,
       3.88697609e-01, 7.92586811e-01, 1.20777583e-01, 3.47123047e-01,
       3.93354746e-01, 1.19625347e-01, 3.20717813e-02, 1.09923267e-01,
       5.87873883e-02, 6.73164357e-01, 6.49541452e-01, 3.06283819e-02,
       1.03979308e-01, 4.10639404e-02, 2.29636926e-01, 6.86837445e-01,
       3.83602958e-03, 2.42023381e-01, 1.36469990e-02, 1.17567580e-01,
       1.54178740e-01, 2.40358353e-01])
```

```
roc_auc_score(y_test,y_prob)
```

```
np.float64(0.7596782302664656)
```

```
# fpr v/s tpr
fpr,tpr,threshold = roc_curve(y_test,y_prob)
```

```
# plot roc curve
plt.figure(figsize=(5,2))
plt.plot(fpr,tpr)
plt.plot([0,1],[0,1])
plt.xlabel('fpr')
plt.ylabel('tpr')
plt.title('roc curve')
plt.show()
```



KNN Classifier

```
from sklearn.neighbors import KNeighborsClassifier
model1 = KNeighborsClassifier(n_neighbors=7)
model1.fit(x_sampled,y_sampled)
y_pred1 = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred1))
print("Accuracy: " ,accuracy_score(y_test,y_pred1))
print('Recall: ',recall_score(y_test,y_pred1))
print('Precision: ',precision_score(y_test,y_pred1))
print('f1_score: ',f1_score(y_test,y_pred1))
```

```
confusion matrix
 [[223  32]
 [ 19  20]]
```

```
Accuracy:  0.826530612244898
Recall:  0.5128205128205128
Precision:  0.38461538461538464
f1_score:  0.43956043956043955
```

Decision Tree Classifier

```
from sklearn.tree import DecisionTreeClassifier
model2 = DecisionTreeClassifier()
model2.fit(x_sampled,y_sampled)
y_pred2 = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred2))
print("Accuracy: " ,accuracy_score(y_test,y_pred2))
print('Recall: ',recall_score(y_test,y_pred2))
print('Precision: ',precision_score(y_test,y_pred2))
print('f1_score: ',f1_score(y_test,y_pred2))
```

```
confusion matrix
 [[223  32]
 [ 19  20]]
Accuracy:  0.826530612244898
Recall:  0.5128205128205128
Precision:  0.38461538461538464
f1_score:  0.43956043956043955
```

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='gini',max_depth=50,max_features='sqrt',splitter='best',min_samples_leaf=5,min_samples
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[213  42]
 [ 28  11]]
Accuracy:  0.7619047619047619
Recall:  0.28205128205128205
Precision:  0.20754716981132076
f1_score:  0.2391304347826087
```

Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier
model3 = RandomForestClassifier()
model3.fit(x_sampled,y_sampled)
y_pred3 = model.predict(x_test)
```

```
# evaluate
print("confusion matrix\n", confusion_matrix(y_test,y_pred3))
print("Accuracy: " ,accuracy_score(y_test,y_pred3))
print('Recall: ',recall_score(y_test,y_pred3))
print('Precision: ',precision_score(y_test,y_pred3))
print('f1_score: ',f1_score(y_test,y_pred3))
```

```
confusion matrix
 [[213  42]
 [ 28  11]]
Accuracy:  0.7619047619047619
Recall:  0.28205128205128205
Precision:  0.20754716981132076
f1_score:  0.2391304347826087
```

```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=10,max_depth=50,
                               bootstrap=True,criterion='gini',
                               max_features='sqrt',min_samples_split=6,
                               min_samples_leaf=5)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[238  17]
 [ 25  14]]
Accuracy:  0.8571428571428571
Recall:  0.358974358974359
Precision:  0.45161290322580644
f1_score:  0.4
```

Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier
model4 = GradientBoostingClassifier()
model4.fit(x_sampled,y_sampled)
y_pred4 = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred4))
print("Accuracy: " ,accuracy_score(y_test,y_pred4))
print('Recall: ',recall_score(y_test,y_pred4))
print('Precision: ',precision_score(y_test,y_pred4))
print('f1_score: ',f1_score(y_test,y_pred4))
```

```
confusion matrix
 [[238  17]
 [ 25  14]]
Accuracy:  0.8571428571428571
Recall:  0.358974358974359
Precision:  0.45161290322580644
f1_score:  0.4
```

```
from sklearn.ensemble import GradientBoostingClassifier
model = GradientBoostingClassifier(n_estimators=10,max_depth=50,
                                   learning_rate=0.01,
                                   max_features='sqrt',min_samples_split=6,
                                   min_samples_leaf=5)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[235  20]
 [ 27  12]]
Accuracy:  0.8401360544217688
Recall:  0.3076923076923077
Precision:  0.375
f1_score:  0.3380281690140845
```

XGBOOST Classifier

```
from xgboost import XGBClassifier
model5 = XGBClassifier()
model5.fit(x_sampled,y_sampled)
y_pred5 = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred5))
print("Accuracy: " ,accuracy_score(y_test,y_pred5))
print('Recall: ',recall_score(y_test,y_pred5))
print('Precision: ',precision_score(y_test,y_pred5))
print('f1_score: ',f1_score(y_test,y_pred5))
```

```
confusion matrix
 [[235  20]
 [ 27  12]]
Accuracy:  0.8401360544217688
Recall:  0.3076923076923077
Precision:  0.375
f1_score:  0.3380281690140845
```

```
from xgboost import XGBClassifier
model = XGBClassifier(n_estimators=50,max_depth=10,
                      learning_rate=0.1, gamma=0.1,subsample=0.8,
                      colsample_bytree=0.8,reg_alpha =0.1 ,
                      reg_lambda=1,n_jobs=-1)
model.fit(x_sampled,y_sampled)
y_pred = model.predict(x_test)
```

```
print("confusion matrix\n", confusion_matrix(y_test,y_pred))
print("Accuracy: " ,accuracy_score(y_test,y_pred))
print('Recall: ',recall_score(y_test,y_pred))
print('Precision: ',precision_score(y_test,y_pred))
print('f1_score: ',f1_score(y_test,y_pred))
```

```
confusion matrix
 [[241  14]
 [ 27  12]]
Accuracy:  0.8605442176870748
Recall:  0.3076923076923077
Precision:  0.46153846153846156
f1_score:  0.36923076923076925
```

K Fold Cross validation

```
from sklearn.model_selection import KFold , cross_val_score
model = XGBClassifier()
kf = KFold(n_splits=5,shuffle=True,random_state=42)
scores = cross_val_score(model,x_sampled,y_sampled,cv=kf)
print(scores)
print(np.mean(scores))
```

```
[0.90816327 0.91560102 0.90792839 0.93861893 0.9028133 ]
0.9146249804269534
```

Stratified Cross validation

```
from sklearn.model_selection import StratifiedKFold
model = XGBClassifier()
sf = StratifiedKFold(n_splits=5,shuffle=True,random_state=42)
scores = cross_val_score(model,x_sampled,y_sampled,cv=sf)
print(scores)
print(np.mean(scores))
```

```
[0.91836735 0.92071611 0.90792839 0.91560102 0.9258312 ]
0.9176888146562974
```

Comparison Table of metrics

```
import pandas as pd
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from xgboost import XGBClassifier

# Re-train models to ensure they are fitted and to correct prediction assignments
# Logistic Regression
model_lr = LogisticRegression(max_iter=60000) # Increased max_iter further to address convergence warning
model_lr.fit(x_sampled, y_sampled)
y_pred_lr = model_lr.predict(x_test)

# KNN Classifier
model1 = KNeighborsClassifier(n_neighbors=7)
model1.fit(x_sampled, y_sampled)
y_pred1 = model1.predict(x_test)

# Decision Tree Classifier
model2 = DecisionTreeClassifier()
model2.fit(x_sampled, y_sampled)
y_pred2 = model2.predict(x_test)

# Random Forest Classifier (using the hyper-parameters from the last execution in the notebook)
model3 = RandomForestClassifier(n_estimators=10,max_depth=50,
                                bootstrap=True,criterion='gini',
                                max_features='sqrt',min_samples_split=6,
                                min_samples_leaf=5)
```

```
model3.fit(x_sampled, y_sampled)
y_pred3 = model3.predict(x_test)

# Gradient Boosting Classifier (using the hyper-parameters from the last execution in the notebook)
model4 = GradientBoostingClassifier()
model4.fit(x_sampled, y_sampled)
y_pred4 = model4.predict(x_test)

# XGBoost Classifier (using the hyper-parameters from the last execution in the notebook)
model5 = XGBClassifier(n_estimators=50,max_depth=10,
                       learning_rate=0.1, gamma=0.1,subsample=0.8,
                       colsample_bytree=0.8,reg_alpha =0.1 ,
                       reg_lambda=1,n_jobs=-1)
model5.fit(x_sampled, y_sampled)
y_pred5 = model5.predict(x_test)


# List to store metrics
comparison_list = []

# Now, `my_models` will correctly refer to the fitted models and their predictions
my_models = [
    ("Logistic Regression", model_lr, y_pred_lr),
    ("KNN", model1, y_pred1),
    ("Decision Tree", model2, y_pred2),
    ("Random Forest", model3, y_pred3),
    ("Gradient Boosting", model4, y_pred4),
    ("XG Boosting", model5, y_pred5)
]

for name, m, p in my_models:
    if hasattr(m, 'predict_proba'):
        y_prob = m.predict_proba(x_test)[:, 1]
    else:
        y_prob = None

    comparison_list.append({
        "Model": name,
        "Accuracy": accuracy_score(y_test, p),
        "Precision": precision_score(y_test, p),
        "Recall": recall_score(y_test, p),
        "F1 Score": f1_score(y_test, p),
        "ROC-AUC": roc_auc_score(y_test, y_prob) if y_prob is not None else float('nan')
    })

comparison_df = pd.DataFrame(comparison_list)

comparison_df.set_index("Model", inplace=True)
# Display the horizontal table
display(comparison_df)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (sta
STOP: TOTAL NO. OF F,G EVALUATIONS EXCEEDS LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Logistic Regression	0.853741	0.456522	0.538462	0.494118	0.772348
KNN	0.588435	0.146552	0.435897	0.219355	0.556461
Decision Tree	0.727891	0.211268	0.384615	0.272727	0.582504
Random Forest	0.863946	0.482759	0.358974	0.411765	0.772298
Gradient Boosting	0.863946	0.487179	0.487179	0.487179	0.772750
XG Boosting	0.860544	0.461538	0.307692	0.369231	0.759175

Next steps: Generate code with comparison_df New interactive sheet

Based on the comparison table:

Logistic Regression: This model performs very well, achieving the highest ROC-AUC (0.772), Recall (0.538), and F1 Score (0.494). It also has a strong Accuracy (0.854).

Gradient Boosting: This model has the highest Accuracy (0.864) and a good F1 Score (0.487).

XGBoost: It also shows good performance with high Accuracy (0.861) and a competitive ROC-AUC (0.762).

F1 Score and ROC-AUC which are often crucial for imbalanced datasets like attrition prediction, Logistic Regression appears to be the most robust and overall best-performing model among those evaluated.

Final Insight

Key Influencing the attrition

```
import matplotlib.pyplot as plt
import pandas as pd
importances = pd.Series(model5.feature_importances_, index=x_sampled.columns)
top_10 = importances.nlargest(10)
print("--- Top 10 Attrition Factors ---")
print(top_10)
plt.figure(figsize=(12, 8))
top_10.plot(kind='barh', color='crimson')
plt.title('Key Factors Influencing Attrition', fontsize=15)
plt.xlabel('Importance Score', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.gca().invert_yaxis()
plt.grid(axis='x', linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

```
--- Top 10 Attrition Factors ---
StockOptionLevel    0.212800
JobLevel            0.122041
Department          0.098436
Gender_Male         0.043142
JobInvolvement      0.039995
Gender_Female       0.039697
```